

JAVA CORE CONCEPTS - EXAM READY QUESTIONS & ANSWERS

A focused long-answer revision sheet for university or diploma-style exams. The answers are written in a clean exam format: definition, explanation, example, and key points.

How to use this PDF: Read the question, understand the answer structure, then practice writing the same idea in your own words in the exam. For long answers, the examiner wants concept clarity, not memorized lines.

SECTION 1 - JAVA FUNDAMENTALS

1. Explain Java, its features, and why it is widely used.

Java is a high-level, object-oriented programming language designed to be simple, portable, secure, and reliable. It was created with the idea of **write once, run anywhere**, which means a Java program can run on different platforms without being rewritten.

Main features:

- Platform independent: Java code is compiled into bytecode and executed by the JVM.
- Object oriented: Programs are built using classes and objects.
- Secure: Java avoids many low-level memory risks and supports runtime checks.
- Robust: Strong exception handling and automatic garbage collection improve reliability.
- Multithreaded: Java supports concurrent execution of tasks.
- Portable: The same bytecode can run on different systems with a JVM.

Why it is used: Java is common in enterprise applications, Android development, web backends, banking systems, and large-scale software because it is stable, maintainable, and platform independent.

2. Explain the Java architecture: JDK, JRE, JVM, and bytecode.

Java programs do not run directly as machine code. First, the source code is written in a **.java** file and compiled by the Java compiler into **bytecode**, which is stored in a **.class** file. Bytecode is an intermediate form of code that can run on any machine that has a JVM.

JDK (Java Development Kit) is used to develop Java programs. It contains the compiler, JRE, debugger, and development tools.

JRE (Java Runtime Environment) provides the libraries and runtime support required to run Java programs.

JVM (Java Virtual Machine) reads bytecode and converts it into machine-specific instructions.

Execution flow: Source code -> Compiler -> Bytecode -> JVM -> Machine code.

Important relation: JDK contains JRE, and JRE contains JVM.

3. Explain data types, variables, type conversion, and type casting in Java.

A **variable** is a named memory location used to store data. Java requires variables to be declared with a data type before use.

Primitive data types: byte, short, int, long, float, double, char, boolean.

Non-primitive types: String, arrays, classes, interfaces, objects.

Type conversion is the automatic conversion of a smaller data type into a larger one, such as int to double.

This is called widening conversion and is safe.

Type casting is explicit conversion, where the programmer forces one data type into another, such as double to int. This may cause loss of data.

Example: `int a = 10; double b = a; // conversion. double x = 9.8; int y = (int)x; // casting.`

In exams, mention that Java is strongly typed, so data type rules are strict and help prevent errors.

4. Compare Java with C and C++.

Java and C/C++ are all powerful languages, but they are built for different goals.

Java vs C:

- Java is object oriented, while C is procedure oriented.
- Java is platform independent because of bytecode and JVM.
- Java has automatic garbage collection; C requires manual memory management.
- Java is safer because it avoids pointer misuse in the usual programming model.

Java vs C++:

- Java does not allow direct multiple inheritance through classes, while C++ does.
- Java has no operator overloading for user-defined operators, but C++ supports it.
- Java separates compile-time and runtime more cleanly through the JVM.

Conclusion: Java sacrifices some low-level control to gain portability, safety, and easier maintenance, which is why it is used widely in real-world software.

5. Explain the storage model of Java using stack, heap, and garbage collection.

Java memory is typically discussed using the **stack** and **heap**.

The **stack** stores method calls, local variables, and function execution details. It works in a last-in, first-out manner. The **heap** stores objects created using the **new** keyword. Multiple references can point to the same heap object.

Garbage collection is the automatic process by which Java removes objects that are no longer reachable. This helps prevent memory leaks and reduces manual memory errors.

Why this matters: Understanding stack and heap helps explain references, object creation, scope, and memory behavior. It also helps answer questions about null pointers, object lifetime, and garbage collection.

SECTION 2 - CLASSES, OBJECTS, AND KEYWORDS

6. Explain classes, objects, and methods with a suitable example.

A **class** is a blueprint or template used to define properties and behaviors. An **object** is a real instance of a class.

A class contains **data members** (variables) and **member functions** (methods). The method defines the behavior, while the variables store the state of an object.

Example: A Student class may contain roll number, name, and marks. Each student created from the class is an object with its own values.

Importance: Classes and objects are the foundation of object-oriented programming. They help model

real-world entities in code, improve reuse, and make large programs easier to manage.

7. Explain constructors in Java. Also differentiate between constructor and method.

A **constructor** is a special member used to initialize objects when they are created. It has the same name as the class and does not return a value.

Types:

- Default constructor
- Parameterized constructor
- Constructor overloading

A constructor is called automatically when an object is created. Its job is initialization, not regular logic.

Constructor vs method:

- Constructor name must match the class; method name can be anything.
- Constructor has no return type; methods have return types.
- Constructor runs automatically; method is called explicitly.
- Constructor is used for initialization; method is used for performing actions.

In exams, write that constructors are essential for object setup and can be overloaded to initialize objects in different ways.

8. Explain the keywords **this**, **super**, **static**, **final**, and **this()** in Java.

this refers to the current object. It is used to resolve ambiguity between instance variables and local variables, and to call another constructor in the same class using **this()**.

super refers to the parent class object. It is used to access parent class variables, methods, and constructors using **super()**.

static means the member belongs to the class, not to a particular object. A static variable is shared by all objects, and a static method can be called without creating an object.

final makes something constant or unchangeable. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be inherited.

These keywords are often asked because they test basic object behavior and memory understanding.

9. Explain access modifiers in Java with their scope.

Java provides four access modifiers to control visibility:

- private** - accessible only inside the same class.
- default** (package-private) - accessible inside the same package.
- protected** - accessible inside the same package and in subclasses.
- public** - accessible from anywhere.

Access control is important because it protects data, supports encapsulation, and prevents unauthorized access.

In exams, say that private gives the highest level of restriction and public gives the widest access.

SECTION 3 - OOP PRINCIPLES

10. Explain encapsulation, abstraction, inheritance, and polymorphism in Java.

These are the four pillars of object-oriented programming.

Encapsulation means wrapping data and methods into a single unit and hiding internal details using access modifiers.

Abstraction means showing only essential features and hiding implementation details.

Inheritance means creating a new class from an existing class to reuse and extend functionality.

Polymorphism means one interface, many forms. The same method name can behave differently in different situations.

Why important? These principles improve code reuse, security, flexibility, and maintainability. In exam answers, give one real-life example for each, such as a car, bank account, or student system.

11. Explain inheritance in Java and its types. Also mention why multiple inheritance is not supported through classes.

Inheritance is a mechanism in which one class acquires the properties and behavior of another class. The existing class is called the **parent** or **super** class, and the new class is called the **child** or **sub** class.

Types of inheritance supported in Java through classes:

- Single inheritance
- Multilevel inheritance
- Hierarchical inheritance

Java does **not** support multiple inheritance through classes because it can create ambiguity, especially when two parent classes contain methods with the same name. This is called the diamond problem.

Java achieves multiple inheritance safely through interfaces.

12. Explain polymorphism. Differentiate between method overloading and method overriding.

Polymorphism means the same name can represent different behaviors. It makes programs flexible and easier to extend.

Method overloading is compile-time polymorphism. It occurs when multiple methods have the same name but different parameter lists in the same class.

Method overriding is runtime polymorphism. It occurs when a child class provides its own implementation of a parent class method.

Overloading: same name, different arguments, same class, compile-time decision.

Overriding: same method signature, different classes, runtime decision.

Dynamic method dispatch happens when a parent reference points to a child object and the overridden method is resolved at runtime.

13. Explain abstract class, abstract method, and interface. Compare abstract class and interface.

An **abstract class** is a class that cannot be instantiated directly. It can contain both abstract and concrete methods. An **abstract method** has no body and must be implemented by the subclass.

An **interface** is a blueprint of behavior. It mainly defines method signatures that implementing classes must

provide.

Abstract class vs interface:

- Abstract class can contain state and constructors; interface normally focuses on behavior.
- A class can extend only one abstract class, but it can implement multiple interfaces.
- Abstract classes are used for partial abstraction; interfaces are used for complete contract-based design.

In modern Java, interfaces are also powerful because they support default and static methods.

14. Explain packages and their importance in Java.

A **package** is a namespace used to group related classes and interfaces. It helps organize code in a structured way.

Advantages of packages:

- Avoid name conflicts
- Improve code organization
- Support access protection
- Make large projects easier to maintain

Java provides built-in packages such as `java.util`, `java.io`, and `java.lang`.

In exams, mention that packages are like folders for classes: they keep related code together and make development cleaner.

SECTION 4 - STRINGS, ARRAYS, AND WRAPPER CLASSES

15. Explain arrays in Java. Differentiate between 1D and 2D arrays.

An **array** is a fixed-size collection of elements of the same data type stored in contiguous memory locations.

1D array: stores data in a single line, such as marks of students.

2D array: stores data in rows and columns, such as a matrix or table.

Arrays are useful when the number of values is known and similar processing is needed for all elements.

Limitations: Arrays have fixed size and can store only same-type elements. For dynamic storage, collections are better.

16. Explain String handling in Java. Compare String, StringBuffer, and StringBuilder.

A **String** is a sequence of characters. In Java, String objects are immutable, which means once created, their content cannot be changed.

Why immutability matters: It improves security, supports string pooling, and makes String objects safe for sharing.

String: immutable, slower for frequent modification.

StringBuffer: mutable and thread-safe, suitable when multiple threads may modify the data.

StringBuilder: mutable but not thread-safe, usually faster than StringBuffer in single-threaded programs.

In exams, clearly mention that String is best for fixed text, while StringBuffer and StringBuilder are used for repeated modifications.

17. Explain the difference between == and equals() in Java.

The **==** operator compares references for objects and compares values for primitive types. The **equals()** method compares actual content, provided the method is properly overridden, which is already done in String.

Example: Two different String objects may contain the same text. In that case, **==** may return false because the references differ, but **equals()** returns true because the content is the same.

This is one of the most common exam traps. For object content comparison, use **equals()**; for reference comparison, use **==**.

18. Explain wrapper classes, autoboxing, and unboxing.

Wrapper classes convert primitive data types into objects. For example, **int** becomes **Integer**, **char** becomes **Character**, and **double** becomes **Double**.

Autoboxing is the automatic conversion of a primitive into its wrapper class object.

Unboxing is the automatic conversion of a wrapper object back into a primitive.

These concepts are useful because many Java utilities and collection classes work with objects, not primitives.

In exam answers, mention that wrapper classes help when primitives need to be stored in collections or used as objects.

SECTION 5 - EXCEPTION HANDLING AND FILE HANDLING

19. Explain exception handling in Java. Also differentiate between checked and unchecked exceptions.

Exception handling is the mechanism used to handle runtime errors gracefully so that the normal flow of the program can continue.

Java uses **try**, **catch**, **finally**, **throw**, and **throws**.

Checked exceptions are checked at compile time. The programmer must handle them, such as **IOException**.

Unchecked exceptions are checked at runtime, such as **ArithmeticException** and **NullPointerException**.

Why important: Exception handling prevents sudden program termination, improves reliability, and helps in writing robust software.

20. Explain try, catch, finally, throw, throws, and user-defined exceptions.

try contains code that may generate an exception. **catch** handles the exception. **finally** executes whether the exception occurs or not, usually for cleanup.

throw is used to explicitly throw an exception object. **throws** is used in a method declaration to indicate that the method may throw exceptions.

A **user-defined exception** is created by the programmer for application-specific error handling.

In exams, explain that **throw** is used inside a method, while **throws** is used in the method signature.

21. Explain file handling in Java. Differentiate between byte streams and character streams.

File handling is used to read and write data stored in files. Java provides stream-based I/O classes for this purpose.

Byte streams handle raw binary data. They are used for images, audio, video, and non-text files.

Character streams handle text data using Unicode characters. They are used for reading and writing text files.

Common classes: FileInputStream, FileOutputStream, FileReader, FileWriter.

In exam answers, mention that byte streams are suitable for binary files, while character streams are suitable for text processing.

22. Explain serialization and deserialization in Java.

Serialization is the process of converting an object into a byte stream so that it can be stored in a file or sent over a network.

Deserialization is the reverse process, where the byte stream is converted back into an object.

These concepts are useful in saving object state and transferring object data between systems.

In exams, note that serialization is important for persistence and communication, and the Serializable interface is used for this purpose.

SECTION 6 - MULTITHREADING, COLLECTIONS, AND JDBC

23. Explain multithreading in Java and the life cycle of a thread.

Multithreading is the execution of multiple threads simultaneously within a program. It improves CPU utilization and makes programs more responsive.

A thread is a lightweight sub-process. Java supports multithreading through the Thread class and the Runnable interface.

Thread life cycle states: New, Runnable, Running, Waiting/Blocked, and Terminated.

Advantages: Better performance, smoother user experience, and parallel task execution.

In exams, mention that multithreading is important in games, servers, and applications that handle multiple tasks at the same time.

24. Explain synchronization and why it is needed in multithreading.

Synchronization is a mechanism used to control access to a shared resource when multiple threads are working at the same time.

Without synchronization, threads may interfere with each other and cause data inconsistency. This is called a race condition.

Java provides synchronized methods and synchronized blocks to make sure only one thread can access the critical section at a time.

In exam answers, connect synchronization with safe shared data access, like bank account transactions or ticket booking systems.

25. Explain the Java Collections Framework. Compare List, Set, and Map.

The **Collection Framework** is a set of interfaces and classes used to store and manipulate groups of objects efficiently.

List stores ordered elements and allows duplicates.

Set stores unique elements and does not allow duplicates.

Map stores key-value pairs where keys are unique.

Common implementations: ArrayList, LinkedList, HashSet, TreeSet, HashMap.

In exams, explain that the collection framework gives ready-made data structures, which save time and improve code quality.

26. Compare ArrayList, LinkedList, HashSet, and HashMap.

ArrayList is a resizable array. It is good for fast random access.

LinkedList stores elements in nodes and is better for frequent insertions and deletions.

HashSet stores unique elements and does not maintain insertion order.

HashMap stores key-value pairs and provides fast access based on keys.

In exam answers, mention that the choice depends on the requirement: ordered list, unique values, or fast key-based lookup.

27. Explain JDBC and its basic steps.

JDBC (Java Database Connectivity) is the API used to connect Java applications with databases.

Basic steps:

1. Load the driver
2. Establish the connection
3. Create a statement
4. Execute the query
5. Process the result
6. Close the connection

JDBC is important because it allows Java programs to store, retrieve, and update data in a database.

In exams, say that JDBC acts as a bridge between Java code and database systems.

SECTION 7 - HIGH-VALUE DIFFERENCES TO MEMORIZE

These compact differences are frequently asked as short notes or comparison questions. Memorize the idea, not just the wording.

- **Constructor vs Method:** Constructor initializes objects; method performs actions.
- **String vs StringBuilder:** String is immutable; StringBuilder is mutable and faster for modifications.
- **Overloading vs Overriding:** Overloading happens in the same class with different parameters; overriding happens in parent-child classes with the same signature.
- **Checked vs Unchecked Exception:** Checked exceptions are compile-time checked; unchecked exceptions are runtime exceptions.
- **Interface vs Abstract Class:** Interface focuses on contract; abstract class supports partial implementation and shared state.
- **== vs equals():** == compares references for objects; equals() compares content.

- **ArrayList vs LinkedList:** ArrayList is better for random access; LinkedList is better for frequent insert/delete.

FAST REVISION ORDER

If you have very little time, revise in this order: JDK-JRE-JVM, classes and objects, constructors, inheritance, polymorphism, abstraction/interfaces, exception handling, strings, multithreading, collections, and JDBC.

That order follows the same logic examiners use: first fundamentals, then OOP, then error handling and advanced topics.